

## 1. Individual contribution and boundaries

My main responsibility is to connect a saved NLP phishing model to a usable backend service. I inspected the exported checkpoint, recreated the hybrid inference architecture, prepared the twelve-feature pipeline, loaded the model and tokenizer, exposed a local Flask endpoint, and integrated explanations into the API response. I tested the endpoint with both suspicious and ordinary email examples. I subsequently built/integrated a Chrome extension prototype that uses the same endpoint to analyse pasted text or an open Gmail message. I used AI assistance to draft and troubleshoot code, then tested changes locally; all claims in this handover require confirmation with the final committed artefacts.

The exported trained weights and original training notebook were not produced merely by building this serving API. Dataset preparation and reported benchmark results must be credited to the correct contributor or repository commit. I should not claim model training, independent test-set evaluation, or verified robustness experiments solely because the checkpoint is present and API requests work.

## 2. Artefact inventory and purpose

| Path in project                                       | Technical content and individual work                                                                                                                           |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| phishing-project/app.py                               | Flask application, one-time predictor construction, GET status route, JSON validation and POST /api/analyse route.                                              |
| phishing-project/predictor.py                         | Text normalisation, twelve NLP features, gated hybrid model definition, strict checkpoint loading, inference, class probabilities and LIME explanation adapter. |
| phishing-project/check_model.py                       | Read-only checkpoint inspection used to compare saved weights with custom layer sizes and saved feature statistics.                                             |
| phishing-project/model/config.json, model_config.json | Backbone and model-serving configuration.                                                                                                                       |
| model/pytorch_model.bin                               | Saved learned weights loaded for inference                                                                                                                      |
| model/tokenizer.json, tokenizer_config.json           | Local tokenizer files used before model inference.                                                                                                              |

|                                                                |                                                                                                                                                    |
|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| model/feature_mean.npy,<br>feature_std.npy, feature_names.json | Training-derived statistics and names for the twelve engineered features.                                                                          |
| model/test_results.json                                        | Exported test-result record, <b>not yet independently verified</b> against the checkpoint and dataset used by the API.                             |
| chrome_extension/manifest.json                                 | Chrome extension metadata, popup and permissions for Gmail/tab access and localhost API access.                                                    |
| chrome_extension/popup.html                                    | Email Detection popup with pasted-email input, read button, analysis button and result/word display.                                               |
| chrome_extension/popup.js                                      | Reads the active open Gmail message when requested, POSTs email text to Flask, renders verdict and signed LIME terms, and handles errors/timeouts. |
| chrome_extension/icons/email-check.svg                         | Optional visual asset only if the downloaded icon was added to the project.                                                                        |

### 3. Technical design and data flow

1. A user pastes email text into the extension or explicitly selects **Read open Gmail email**. The extension attempts to extract the subject and the last visible message body from the active Gmail page. Gmail DOM selectors may change; the extracted text is shown for user review before sending.
2. Selecting **Analyse email** sends a JSON body with text to <http://127.0.0.1:5000/api/analyse>. The popup does not silently send text merely because a Gmail tab is open.
3. Flask checks that the body is a JSON object and text is a non-empty string. The predictor instance has already been created at startup, so the large checkpoint is not reloaded for each request.
4. predictor.py applies its email normalisation, extracts twelve NLP features, scales them using saved mean and standard deviation, and tokenises text with the local tokenizer. The model consumes tokens/attention mask plus scaled features.
5. PyTorch inference produces two class probabilities. The served code displays label 0 as legitimate and label 1 as phishing; the exported checkpoint's training label mapping

must still be cross-checked. The response includes the predicted verdict and score for that verdict.

6. The LIME text explainer perturbs the email, calls the same model-prediction function on nearby examples, and fits a local surrogate. Its signed term weights explain the selected prediction locally, not whether the email is truly fraudulent.
7. Flask returns JSON; the extension displays a verdict, score, explanatory terms and connection/validation errors. The extension itself does not train or fine-tune the model.

This is a hybrid text-and-feature system, not a plain BERT text-only model. Any changes to normalisation, feature extraction, feature order, tokenizer, max length or model layer dimensions could make serving inconsistent with training.

## 4. Checkpoint inspection and model compatibility

The checkpoint inspection reported mean and standard-deviation arrays of shape (12,). Saved custom-layer shapes were: context projection (256, 1024), feature projection (256, 12), gate (256, 512), and classifier (2, 256). These observations informed the recreation of the gated-fusion architecture. The predictor loads local files, checks feature array shapes and numerical validity, calls `load_state_dict(..., strict=True)`, and uses `eval()` and inference mode to avoid training behaviour on requests.

Strict weight loading checks structural agreement; it does **not** establish that preprocessing reproduces training, that class labels are correctly interpreted, or that the model is accurate. The saved `model_config.json` reported a text max length of 512 in earlier project work; include an excerpt of the actual final configuration when submitting.

## 5. API contract and validation

| Operation         | Input                                                        | Expected behaviour                                                                                 |
|-------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| GET /             | No email input                                               | Plain status message Phishing API is running. This is only a indication.                           |
| POST /api/analyse | JSON object with a non-empty string text                     | Predicts email class and returns JSON with <code>label_id</code> , verdict, score and explanation. |
| Invalid JSON/body | Missing object, missing text, empty string or incorrect type | HTTP 400 with a JSON error, as implemented in <code>app.py</code> .                                |

Example request:

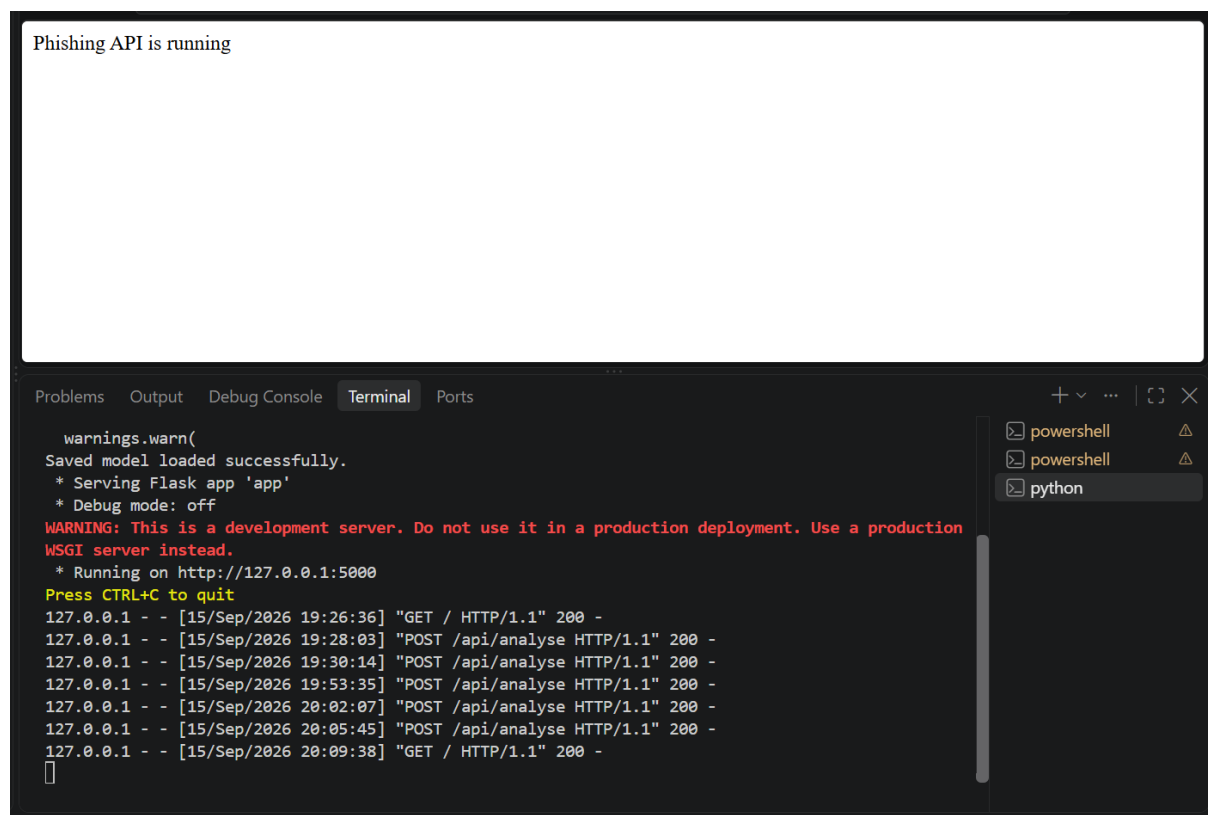
```
{"text": "Urgent: Your account will be suspended today. Submit your password immediately."}
```

Observed explanatory response structure includes explanation.method: "LIME" and a list of terms with word and weight. Copy one **actual response from the final running code** into the submission to document the precise complete schema; field names such as terms, class\_explained and num\_samples depend on the final LIME method. Do not invent fields from an earlier code suggestion.

The API runs on localhost for development, not as a production mail gateway. Browser host permissions allow the extension to call that localhost endpoint. No external deployment, authentication, encrypted transport or mail-gateway policy integration is claimed here.

## 6. Explainability evolution and interpretation

An initial explanation method removed individual words and measured the change in the original class score. It checked only a limited beginning portion of each email; the API described this as **word-removal sensitivity**, an approximate local explanation. It is not SHAP or LIME.

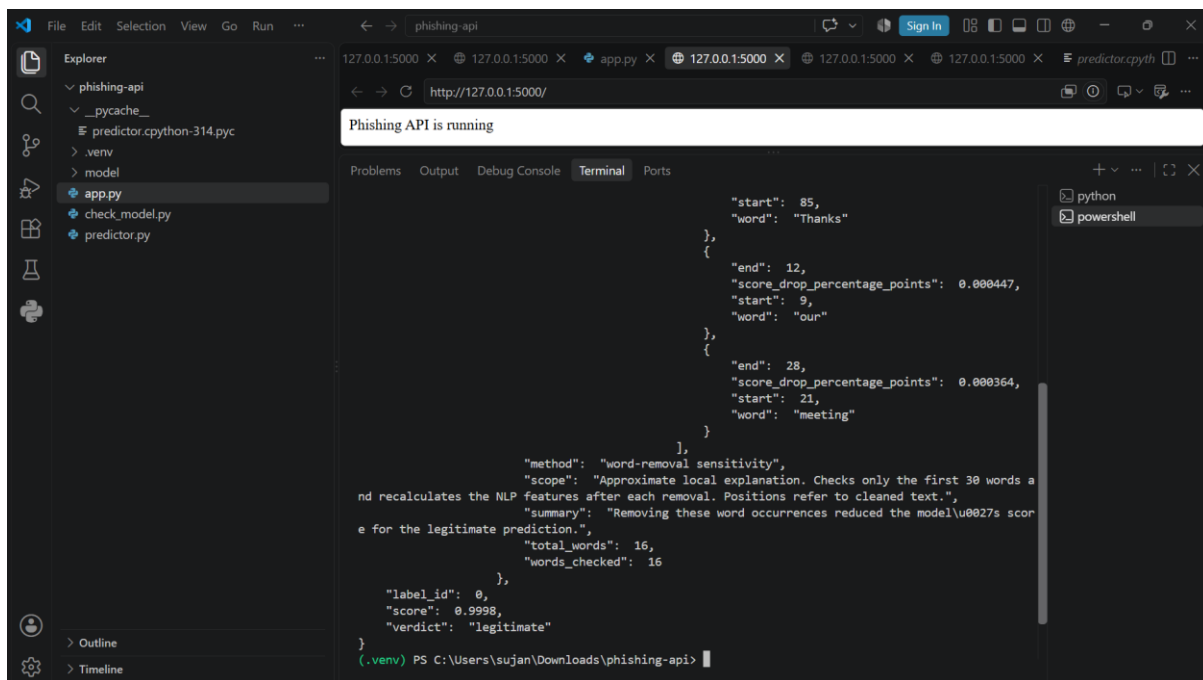


The screenshot shows a web browser window at the top with the text "Phishing API is running". Below it is a terminal window with the following output:

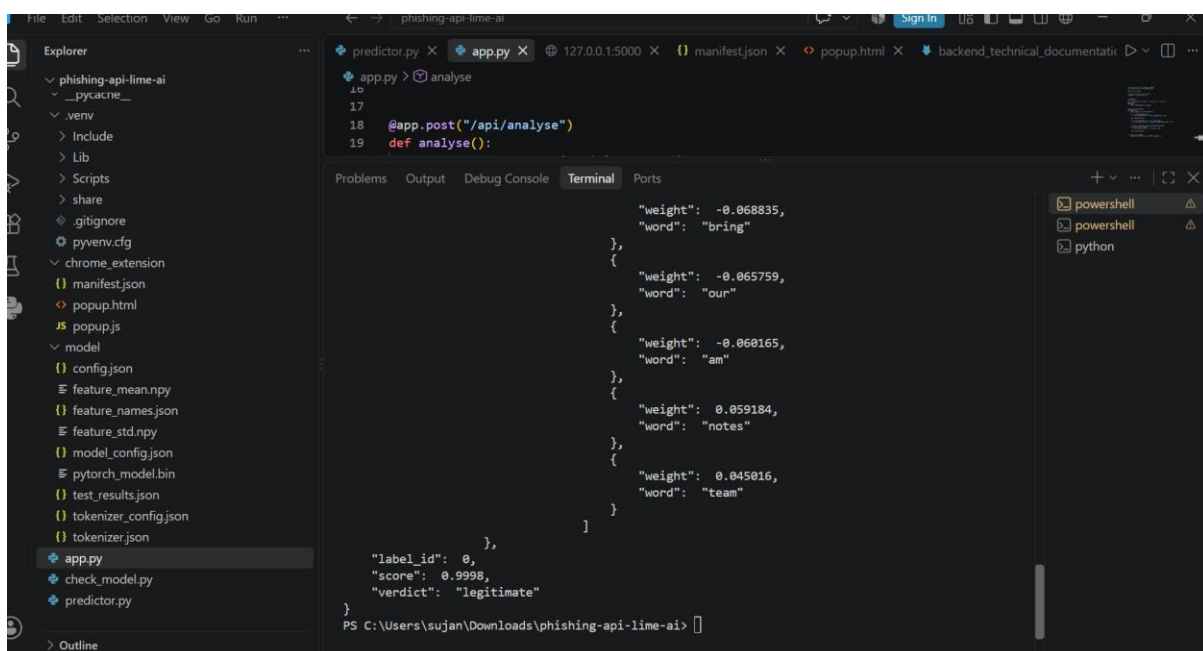
```
Problems  Output  Debug Console  Terminal  Ports

warnings.warn(
Saved model loaded successfully.
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [15/Sep/2026 19:26:36] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 19:28:03] "POST /api/analyse HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 19:30:14] "POST /api/analyse HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 19:53:35] "POST /api/analyse HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 20:02:07] "POST /api/analyse HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 20:05:45] "POST /api/analyse HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 20:09:38] "GET / HTTP/1.1" 200 -
█
```

On the right side of the terminal window, there is a list of open files: powershell, powershell, and python.



The later version installed lime and connected a LimeTextExplainer to a function returning **both class probabilities** for perturbed texts. The hybrid predictor recomputes text tokens and twelve NLP features for each perturbation. The API now identifies the method as LIME and returns word/weight pairs. Positive weights support the **class being explained**; negative weights oppose it. A weight such as -0.068835 is not a 6.9% error rate or negative probability. Terms are a local approximation sensitive to perturbation sampling and the model; they are not proof of malicious intent.



## Development and troubleshooting record

| Problem encountered                                                     | Diagnostic evidence                                                                                            | Change and learning                                                                                                                                      |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Editor import warnings and dependency setup                             | Project environment and import check printed Libraries ready.                                                  | Installed Flask, PyTorch, Transformers, NumPy and SentencePiece and selected the project's Python interpreter. LIME was installed later.                 |
| Saved model not compatible with a simple text-only script               | check_model.py revealed twelve features and custom layer dimensions.                                           | Recreated the hybrid architecture and used saved feature statistics instead of assuming a vanilla DeBERTa classifier.                                    |
| Python indentation/undefined-variable warnings after adding explanation | VS Code showed an expected indented block and undefined text, max_words, top_k.                                | Kept methods at the correct indentation inside PhishingPredictor, then ran py_compile successfully. Syntax compilation alone did not test Flask runtime. |
| HTTP 500 on the explanation endpoint                                    | Flask traceback: PhishingPredictor had no predict attribute when predict_with_explanation called self.predict. | Restored/positioned the prediction method inside the class and retested the endpoint rather than treating a successful status page as proof.             |
| Result format was raw JSON in terminal/status page                      | Screenshots showed curly-brace output and a plain GET / message.                                               | Built an extension popup for human-facing input and rendering while preserving machine-readable API JSON.                                                |
| Explainability technique did not match intended LIME use                | Word-removal output said word-removal sensitivity.                                                             | Installed LIME, added a probability adapter and checked for method: LIME in API output.                                                                  |
| Fictional shipping notice false positive                                | API returned phishing score 0.8481 for an expected legitimate sample.                                          | Documented it as an error and a reason to evaluate false positives and data distribution, not as a passing test.                                         |

## 7. Functional tests observed

| Test message (fictional)                          | Expected   | API verdict observed | Reported score | Interpretation                                       |
|---------------------------------------------------|------------|----------------------|----------------|------------------------------------------------------|
| Account suspension and immediate password request | Phishing   | Phishing             | 0.9981         | Integration behaved as expected on this sample.      |
| Project meeting reminder                          | Legitimate | Legitimate           | 0.9998         | Integration behaved as expected on this sample.      |
| Order-dispatch notice requiring no action         | Legitimate | Phishing             | 0.8481         | False positive; model behaviour needs investigation. |
| Reminder never to share passwords                 | Legitimate | Legitimate           | 0.9996         | Integration behaved as expected on this sample.      |

These manually written messages are **functional checks**, not an independent confusion matrix. Their model scores are not calibrated assurances. The extension was also shown loaded in Chrome under the name **Email Detection**; its files are manifest.json, popup.html and popup.js. A loaded extension card verifies installation only. For in-place integration, capture the **Read open Gmail email** step, the text reviewed in the popup, the final verdict and LIME words, plus the Flask log for the same request. Manual paste tests verify the API/UI path but not Gmail DOM extraction.

## 8. Reproducibility and security handover

To reproduce the serving demonstration on my computer:

1. Open the phishing-api-lime-ai project in VS Code; check that app.py, predictor.py, check\_model.py and the required model/ files are present.
2. Use the project Python environment with the installed Flask, PyTorch, Transformers, NumPy, SentencePiece and LIME packages. Record exact tested versions in a dependency file before handover.
3. In a terminal at project root run python app.py; wait for http://127.0.0.1:5000. Leave the server terminal running.
4. In a separate PowerShell terminal or the extension popup, send a non-empty JSON text value to POST /api/analyse. Check verdict, score, explanation.method and terms; inspect the Flask traceback if HTTP 500 occurs.
5. Load **only** the chrome\_extension subfolder through chrome://extensions > Developer mode > Load unpacked. Confirm manifest.json points default\_popup to popup.html, popup.html loads popup.js, and permissions allow the intended Gmail access and http://127.0.0.1:5000/\*.
6. Use fictional/de-identified email samples. Click Analyse deliberately; do not place private mailbox contents, credentials or a local .venv into public evidence or GitHub. If sharing the checkpoint, document its source and any licence/size restrictions.

The extension uses Gmail page selectors rather than an official mail integration. It may extract the last visible message in a thread and needs a manual-review fallback. The local backend and unrestricted extension host access are suitable for a controlled prototype, not a claim of production deployment.



## Appendix: A

This appendix A shows that chrome extension can read email from your Gmail account and analyse an email whether it is legitimate or phishing.

### Email Detection

Phishing email detection

#### Analyse an email

Read the currently Gmail message or paste a test email below.

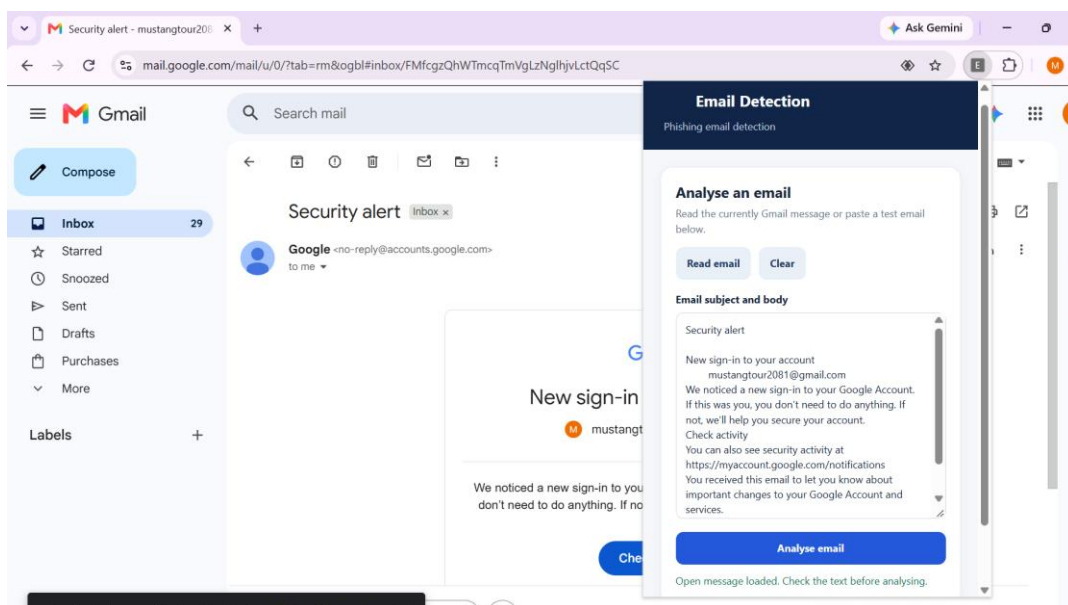
Read emailClear

Email subject and body

Open an email in Gmail and select Read, or paste a de-identified sample here.

Analyse email

Text cleared.



Email Detection

Phishing email detection

Analyse an email

Read the currently Gmail message or paste a test email below.

Read email

Clear

Email subject and body

Security alert  
New sign-in to your account  
mustangtour2081@gmail.com  
We noticed a new sign-in to your Google Account.  
If this was you, you don't need to do anything. If not, we'll help you secure your account.  
Check activity  
You can also see security activity at  
<https://myaccount.google.com/notifications>  
You received this email to let you know about important changes to your Google Account and services.  
© 2026 Google LLC, 1600 Amphitheatre Parkway,

Analyse email

Analysis complete.

Analysis result

Predicted legitimate

99.6%

Model score for the displayed verdict—not a guarantee.

Why this result?  
LIME explains the legitimate verdict. Positive weights support it; negative weights oppose it. Weights are not percentages.

|              |        |
|--------------|--------|
| Google       | +0.316 |
| You          | +0.186 |
| alert        | +0.170 |
| anything     | +0.140 |
| this         | +0.140 |
| Amphitheatre | +0.123 |
| Check        | +0.119 |
| we           | +0.003 |

Email text is sent only when you select Analyse; avoid real personal information during testing.

## Appendix B

It shows we can copy paste our email in extension and check whether it is phishing or legitimate.

### Email Detection

Phishing email detection

#### Analyse an email

Read the currently Gmail message or paste a test email below.

Read emailClear

**Email subject and body**

Urgent: your university account will be suspended today. Reply with your username and password immediately to keep access

Analyse email

Analysis complete.

#### Analysis result

Potential phishing

## 93.4%

Model score for the displayed verdict—not a guarantee.

#### Why this result?

LIME explains the phishing verdict. Positive weights support it; negative weights oppose it. Weights are not percentages.

|            |        |
|------------|--------|
| account    | +0.179 |
| your       | +0.138 |
| university | -0.117 |
| today      | -0.108 |
| password   | +0.087 |
| username   | -0.085 |
| and        | -0.057 |
| keep       | -0.055 |

Email text is sent only when you select Analyse; avoid real personal information during testing.

Email Detection

Phishing email detection

Analyse an email

Read the currently Gmail message or paste a test email below.

Read email

Clear

Email subject and body

Congratulations! You have won a cash prize. Send your bank login details and pay a processing fee today to claim it.

Analyse email

Analysis result

Potential phishing

99.6%

Model score for the displayed verdict—not a guarantee.

Why this result?

LIME explains the phishing verdict. Positive weights support it; negative weights oppose it. Weights are not percentages.

|                 |        |
|-----------------|--------|
| a               | +0.257 |
| claim           | +0.208 |
| prize           | -0.159 |
| Congratulations | -0.154 |
| Send            | -0.142 |
| fee             | +0.129 |
| cash            | +0.122 |
| You             | +0.076 |

Email text is sent only when you select Analyse; avoid real personal information during testing.